

The Two Gradient Problems Need Different Responses

Clipping bounds unstable updates; gated state paths support longer-range credit assignment.

EXPLODING GRADIENTS

Clip the gradient norm

$$g \leftarrow g \min\left(1, \frac{\tau}{\|g\|_2}\right)$$



oversized gradient $\rightarrow$ bounded update

Direction is preserved; the norm is capped at τ . Memory is unchanged.

VANISHING GRADIENTS

Build an additive carry path



A near-identity route can retain state and credit.

GRUs and LSTMs learn when to preserve or replace information.

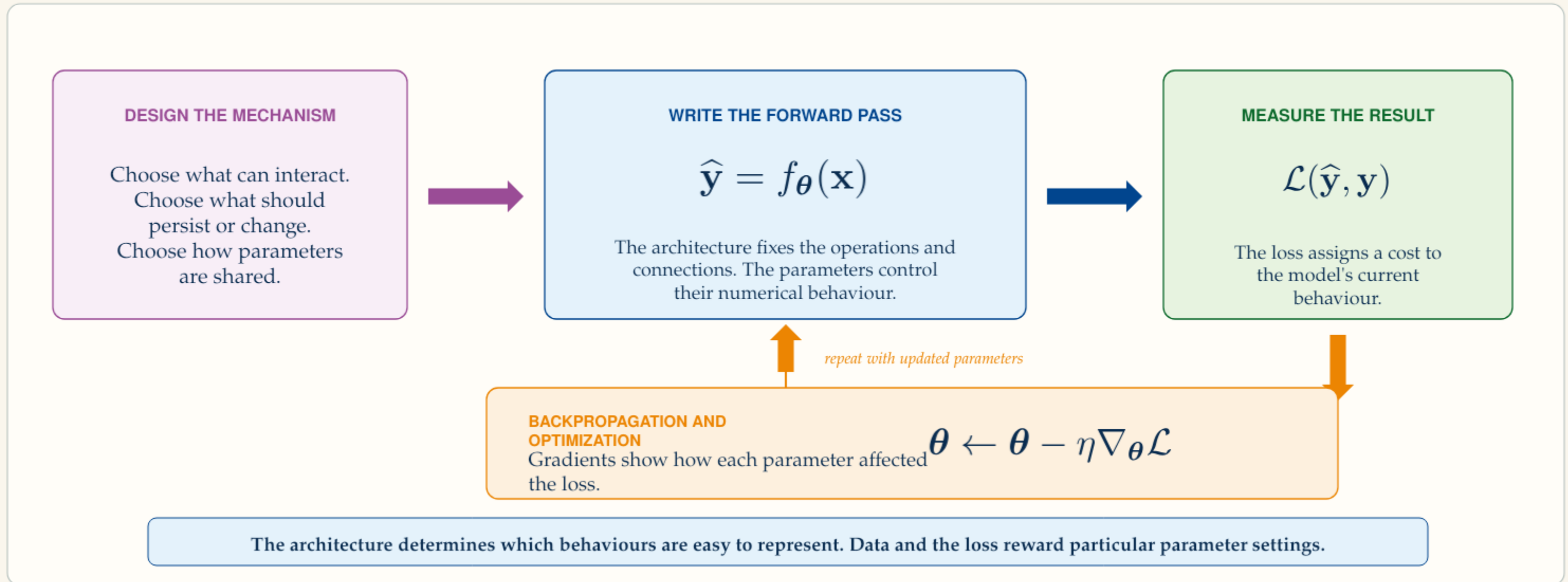
TBPTT is a computational compromise—not a cure for vanishing gradients.

Next: replace a fixed carry weight with a learned gate.

The Deep Learning Design Mindset

Architectures encode a hypothesis about useful computation. Training learns the numerical settings.

FROM AN INDUCTIVE BIAS TO A TRAINED MODEL

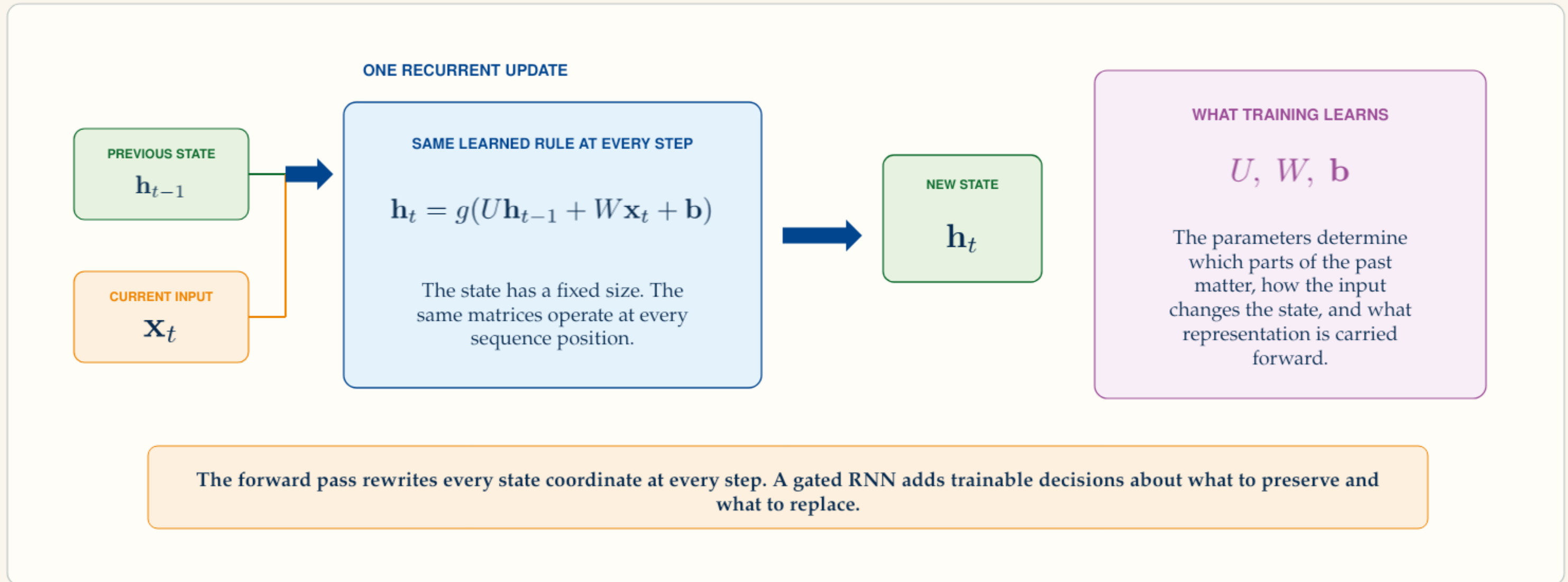


Use this lens for MLPs, CNNs, RNNs, gated RNNs, and attention.

Vanilla RNN: A Trainable State Update

The forward pass assumes that one shared recurrent rule can carry information through a sequence.

THE ARCHITECTURE PROPOSES A MEMORY MECHANISM



The GRU turns state preservation and replacement into context-dependent learned decisions.

A Fixed Carry Weight Makes the State Change Gradually

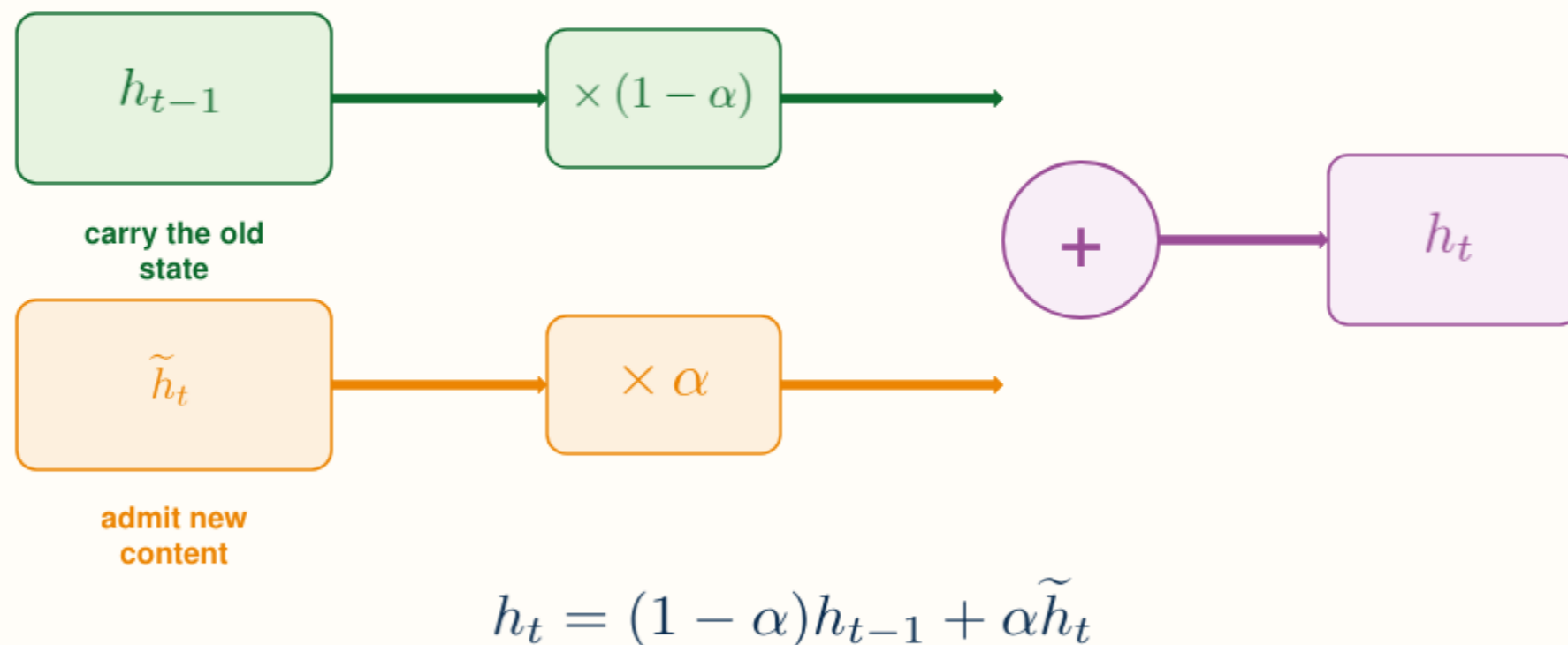
A hand-chosen mixture carries most of the previous state and admits a small amount of the candidate.

A HAND-DESIGNED ADDITIVE CARRY PATH

Candidate = vanilla-RNN state update

Compute the familiar state proposal before mixing.

$$\tilde{h}_t = g(Uh_{t-1} + Wx_t + b)$$



Choose $\alpha = 0.1$

90% is carried forward;
10% is refreshed.

DL callback: fixed momentum / EMA coefficient

If the candidate contributes zero:

$$1.00 \rightarrow 0.90 \rightarrow 0.81 \\ \rightarrow 0.73 \rightarrow 0.66 \rightarrow 0.59$$

The direct carry contribution after k steps

$$h_{t+k}^{\text{carry}} = (1 - \alpha)^k h_t = 0.9^k h_t$$

Smaller α means slower change—and
a longer fixed timescale.

A fixed α cannot decide which information to retain or when to replace it.

The Carry Weight Can Depend on the Current Context

A learned gate reads the previous state and current input, then chooses how much to update.

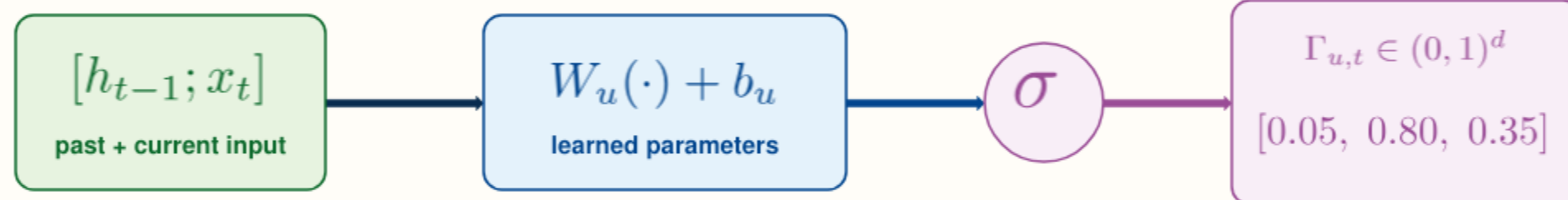
REPLACE THE CONSTANT WITH A LEARNED VECTOR

Fixed carry weight

$$\alpha = 0.1$$

Context-dependent update gate

$$\Gamma_{u,t} = \sigma(W_u[h_{t-1}; x_t] + b_u)$$



Same vector length d : one gate value for each state coordinate.

$$h_{t-1}, \tilde{h}_t, h_t \in \mathbb{R}^d, \quad \Gamma_{u,t} \in (0, 1)^d$$

$$h_t = (1 - \Gamma_{u,t}) \odot h_{t-1} + \Gamma_{u,t} \odot \tilde{h}_t$$

Here: gate ≈ 0 retains; gate ≈ 1 writes the candidate.

What the gate means

$\Gamma \approx 0$ retain the old state

$\Gamma \approx 1$ write the candidate

Intermediate values blend old and new

The parameters are learned by backpropagation through the sequence.

The model now chooses how much to update at each time step and in each state coordinate.

A Simplified GRU Learns What to Write

The vanilla-RNN candidate is blended with the previous state using the learned update gate.

THREE COMPUTATIONS AT EACH TIME STEP

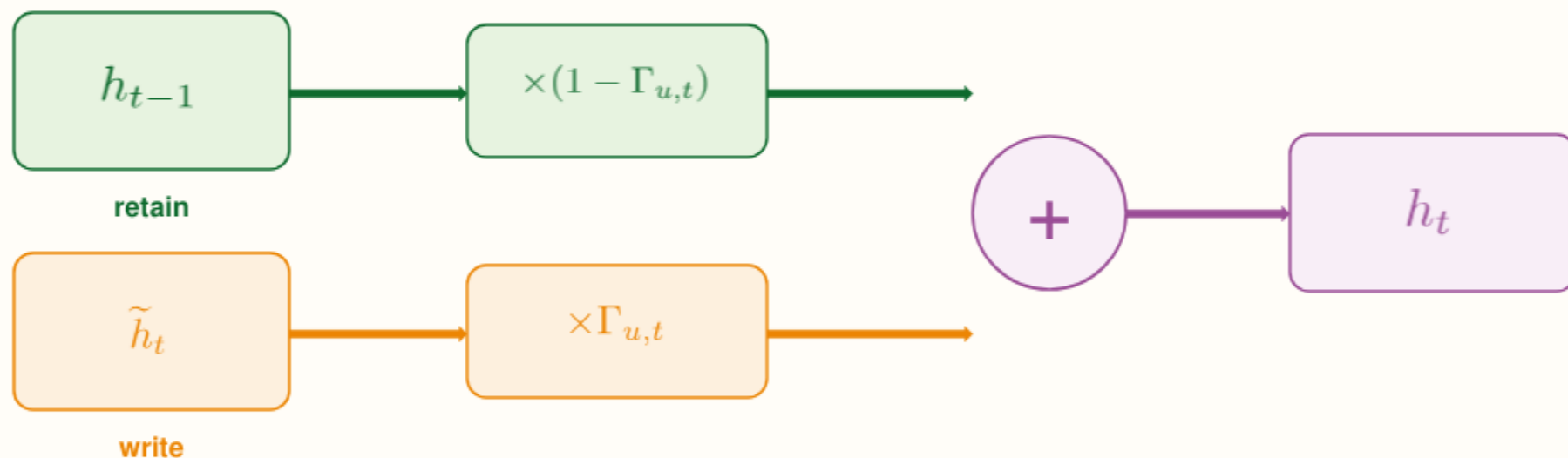
1 Propose

$$\tilde{h}_t = g(Uh_{t-1} + Wx_t + b_h)$$

vanilla-RNN update

2 Choose how much to write

$$\Gamma_{u,t} = \sigma(W_u[h_{t-1}; x_t] + b_u)$$



$$h_t = (1 - \Gamma_{u,t}) \odot h_{t-1} + \Gamma_{u,t} \odot \tilde{h}_t$$

Zoom In on One State Coordinate

Coordinate j has a nearly closed gate.

$$h_{t-1}^{(j)} = 0.8$$

$$\tilde{h}_t^{(j)} = -0.2$$

$$\Gamma_{u,t}^{(j)} = 0.1$$

$$\begin{aligned} h_t^{(j)} &= 0.9(0.8) + 0.1(-0.2) \\ &= 0.72 - 0.02 = 0.70 \end{aligned}$$

The same calculation runs in parallel for every coordinate of the state vector.

The update gate decides when to write; the full GRU also controls how the candidate reads the past.

GRU: Trainable Read and Write Decisions

The forward pass separates candidate construction from the final state update.

THE ARCHITECTURE ENCODES TWO QUESTIONS

READ DECISION

Which coordinates of the previous state should shape the candidate?

WRITE DECISION

How much of the candidate should replace the previous state?

FORWARD PASS

$$\Gamma_{r,t} = \sigma(W_r[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_r)$$

$$\tilde{\mathbf{h}}_t = \tanh(W_h[\Gamma_{r,t} \odot \mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_h)$$

$$\Gamma_{u,t} = \sigma(W_u[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_u)$$

$$\mathbf{h}_t = (\mathbf{1} - \Gamma_{u,t}) \odot \mathbf{h}_{t-1} + \Gamma_{u,t} \odot \tilde{\mathbf{h}}_t$$

Both gates are vectors, so different state coordinates can make different decisions.

WHAT TRAINING LEARNS

The matrices learn when each gate should open or close from the current input and previous state.

The task loss supplies the evidence. No instructor assigns the gate values.

W_r, W_h, W_u

The GRU keeps one state. The LSTM separates persistent memory from the visible state.

The Reset Gate Controls How the Candidate Reads the Past

The full GRU filters the previous state before using it to construct the new candidate.

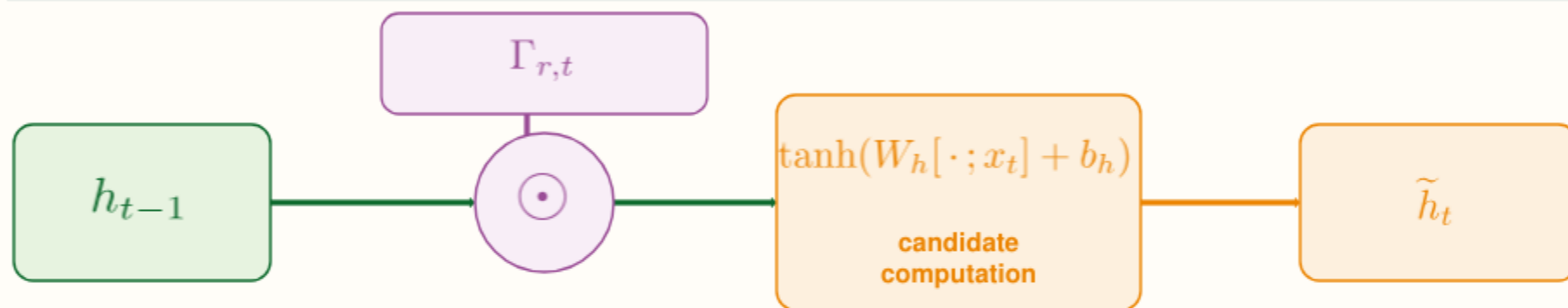
THE CANDIDATE PATH GAINS A SECOND GATE

1 Decide what to read

$$\Gamma_{r,t} = \sigma(W_r[h_{t-1}; x_t] + b_r)$$

2 Form the candidate

$$\tilde{h}_t = \tanh(W_h[\Gamma_{r,t} \odot h_{t-1}; x_t] + b_h)$$



The candidate is the vanilla-RNN update, now computed from a gated previous state.

$$h_{t-1}, \tilde{h}_t \in \mathbb{R}^d, \quad \Gamma_{r,t} \in (0, 1)^d$$

Reset gate is also called the relevance gate in Andrew Ng's presentation.

What the Reset Gate Means

$\Gamma_{r,t}^{(j)} \approx 1$ consult that part of the past

$\Gamma_{r,t}^{(j)} \approx 0$ ignore it while forming the candidate

The reset gate changes what is read.
The update gate still decides what is written.

The reset gate controls candidate formation; the update gate controls the final state blend.

A Full GRU Makes Two Separate Decisions

The reset gate controls candidate formation; the update gate controls how the state is blended.

FOUR COMPUTATIONS AT EACH TIME STEP

- 1 Read gate** $\Gamma_{r,t} = \sigma(W_r[h_{t-1}; x_t] + b_r)$
- 2 Candidate** $\tilde{h}_t = \tanh(W_h[\Gamma_{r,t} \odot h_{t-1}; x_t] + b_h)$
- 3 Write gate** $\Gamma_{u,t} = \sigma(W_u[h_{t-1}; x_t] + b_u)$
- 4 New state** $h_t = (1 - \Gamma_{u,t}) \odot h_{t-1} + \Gamma_{u,t} \odot \tilde{h}_t$

$h_{t-1}, \tilde{h}_t, h_t \in \mathbb{R}^d, \quad \Gamma_{r,t}, \Gamma_{u,t} \in (0, 1)^d$
Both gates have the same vector length as the recurrent state.

Two Gates, Two Locations



What should the candidate read from the previous state?



How much of the candidate should be written?

Both decisions depend on the current context.

The reset gate acts inside the candidate path; the update gate acts at the final blend.

GRU Gates Act Coordinate by Coordinate

Different parts of the state can be consulted, retained or replaced by different amounts.

AN ILLUSTRATIVE THREE-COORDINATE STATE

State information	$\Gamma_{r,t}$	$\Gamma_{u,t}$	Result
Ongoing topic	1.0	0.0	consult and preserve
Subject number	0.9	0.1	mostly preserve
Local phrase pattern	0.1	0.9	mostly replace

The labels are intuition only: real recurrent representations are distributed across many learned coordinates.

$$\Gamma_{r,t}, \Gamma_{u,t} \in (0, 1)^d \quad \text{and} \quad h_t \in \mathbb{R}^d$$

One Value per Coordinate

Read from the past

$$\Gamma_{r,t} = \begin{bmatrix} 1.0 \\ 0.9 \\ 0.1 \end{bmatrix}$$

Write into the state

$$\Gamma_{u,t} = \begin{bmatrix} 0.0 \\ 0.1 \\ 0.9 \end{bmatrix}$$

The two vectors need not agree.

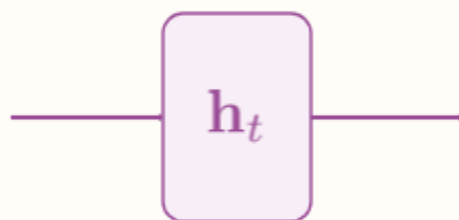
A GRU uses one state for both memory and output; an LSTM separates these two roles.

An LSTM Separates Memory from the Visible State

An LSTM carries a persistent memory state while exposing a separate hidden state.

SEPARATING PERSISTENT MEMORY FROM THE VISIBLE STATE

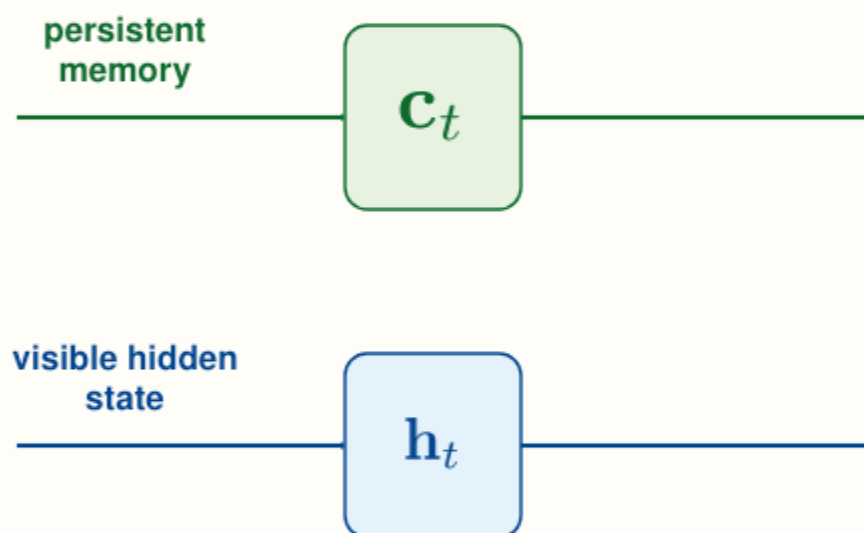
The GRU State



The same vector must carry information forward and be the visible representation.

memory + output

The LSTM State Pathways



Candidate memory
 $\tilde{c}_t = \tanh(W_c[h_{t-1}; x_t] + b_c)$

Same nonlinear state calculation as a vanilla RNN.

Roles of c_t and h_t

c_t

carry information
through time

h_t

reveal what is useful
now

$c_t, h_t \in \mathbb{R}^{d_h}$

Same dimensionality;
different roles.

Next, two learned gates decide what old memory to retain and what new candidate to write.

Forget and Update Gates Control the Memory

Each coordinate independently decides how much old memory to retain and how much candidate memory to write.

THE MEMORY STATE IS AN ADDITIVE MIXTURE

Three learned quantities

Forget gate

$$\Gamma_{f,t} = \sigma(W_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f)$$

Update gate

$$\Gamma_{u,t} = \sigma(W_u[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_u)$$

Candidate

$$\tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_c)$$

Memory update

$$\mathbf{c}_t = \Gamma_{f,t} \odot \mathbf{c}_{t-1} + \Gamma_{u,t} \odot \tilde{\mathbf{c}}_t$$

$$\Gamma_{f,t}, \Gamma_{u,t} \in (0, 1)^{d_h}, \quad \mathbf{c}_t, \tilde{\mathbf{c}}_t \in \mathbb{R}^{d_h}$$

The two gate vectors are learned independently: their coordinates need not sum to one.

One Coordinate

Illustrative memory calculation

Old memory

$$c_{t-1}^{(j)} = 0.8$$

Candidate memory

$$\tilde{c}_t^{(j)} = -0.3$$

$$\Gamma_{f,t}^{(j)} = 0.95, \quad \Gamma_{u,t}^{(j)} = 0.10$$

$$c_t^{(j)} = 0.95(0.8) + 0.10(-0.3) = 0.73$$

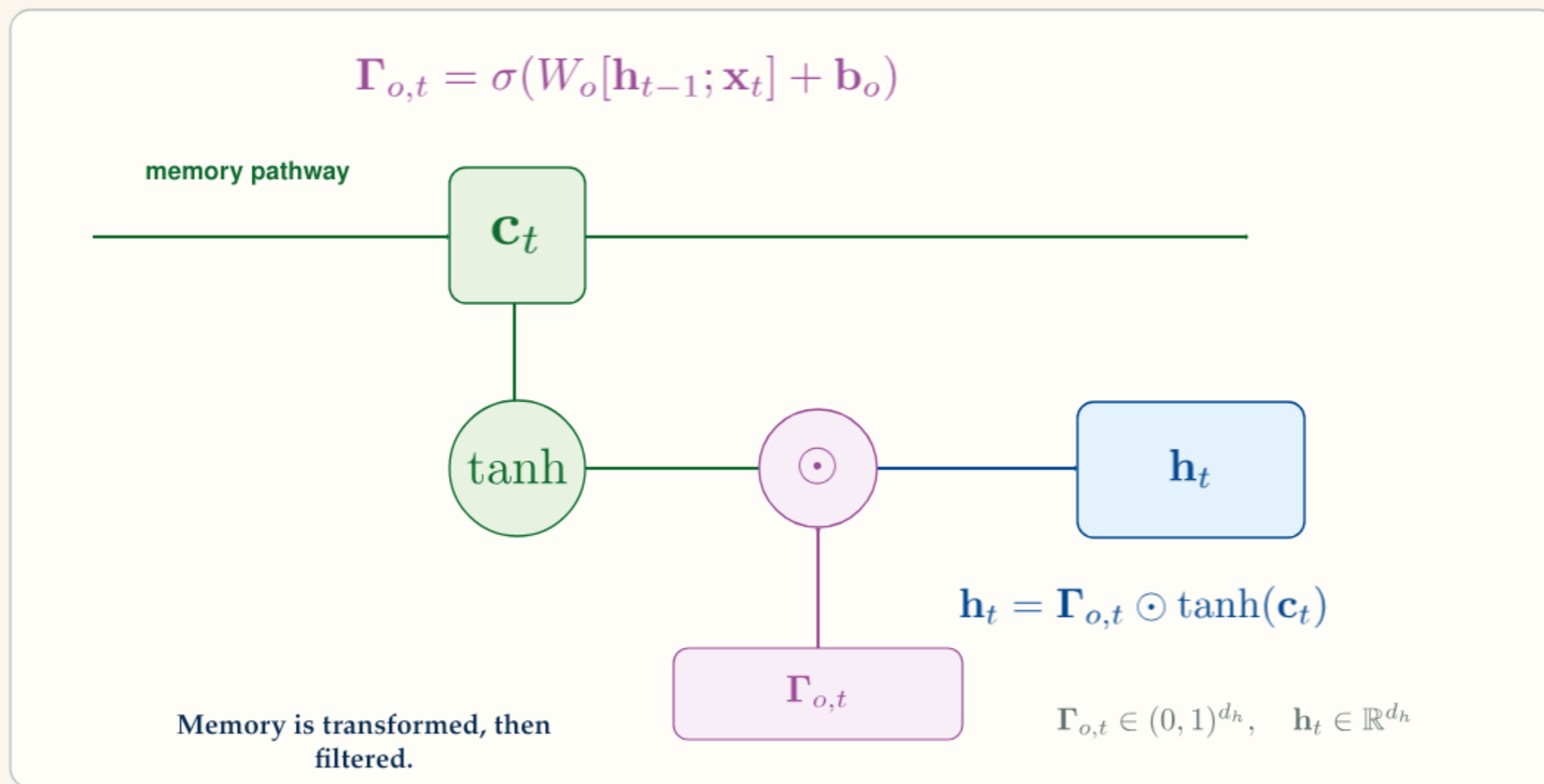
Mostly retain; write only a little.

The memory is updated; the output gate now decides how much of that memory becomes visible.

The Output Gate Controls What the LSTM Reveals

The memory can persist internally even when only part of it is exposed as the hidden state.

MEMORY CONTINUES; THE VISIBLE STATE BRANCHES OFF



Language Intuition

"The keys to the cabinet ..."

Memory state

retain that the subject is plural

Hidden state

reveal it when predicting the verb

A stored feature need not be exposed at every word.

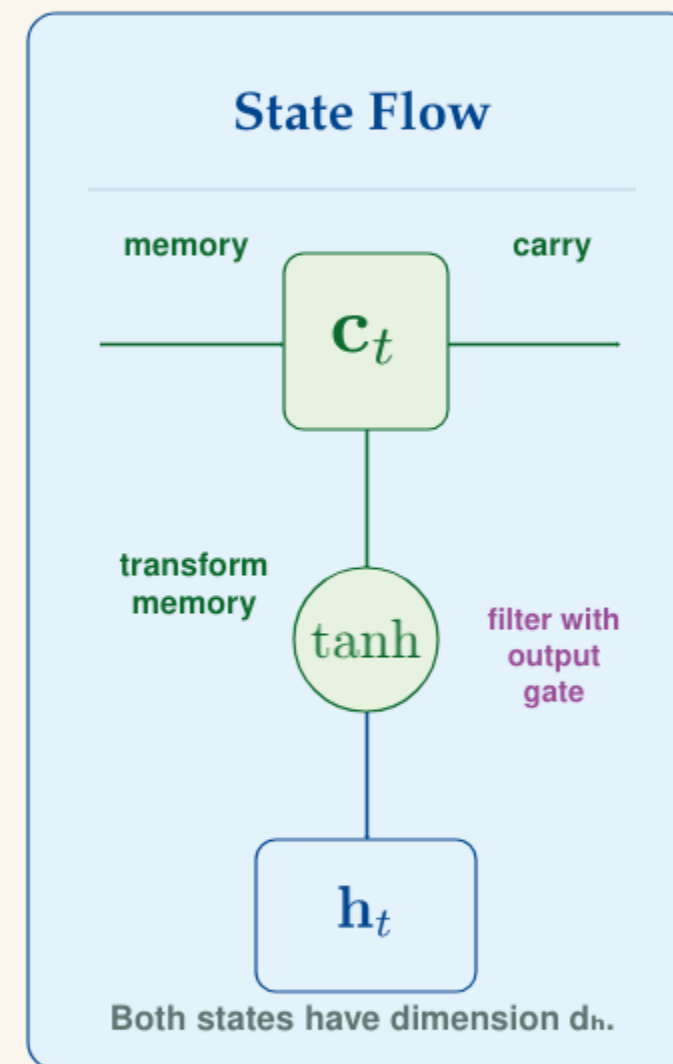
With forget, update and output gates in place, we can assemble the complete LSTM cell.

The Complete LSTM Cell

The gates and candidate jointly update the memory state and produce the visible hidden state.

ONE COMPLETE TIME STEP

Shared recurrent input	$\mathbf{q}_t = [\mathbf{h}_{t-1}; \mathbf{x}_t]$
Forget gate	$\Gamma_{f,t} = \sigma(W_f \mathbf{q}_t + \mathbf{b}_f)$
Update gate	$\Gamma_{u,t} = \sigma(W_u \mathbf{q}_t + \mathbf{b}_u)$
Candidate	$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{q}_t + \mathbf{b}_c)$
Output gate	$\Gamma_{o,t} = \sigma(W_o \mathbf{q}_t + \mathbf{b}_o)$
Memory update	$\mathbf{c}_t = \Gamma_{f,t} \odot \mathbf{c}_{t-1} + \Gamma_{u,t} \odot \tilde{\mathbf{c}}_t$
Visible state	$\mathbf{h}_t = \Gamma_{o,t} \odot \tanh(\mathbf{c}_t)$

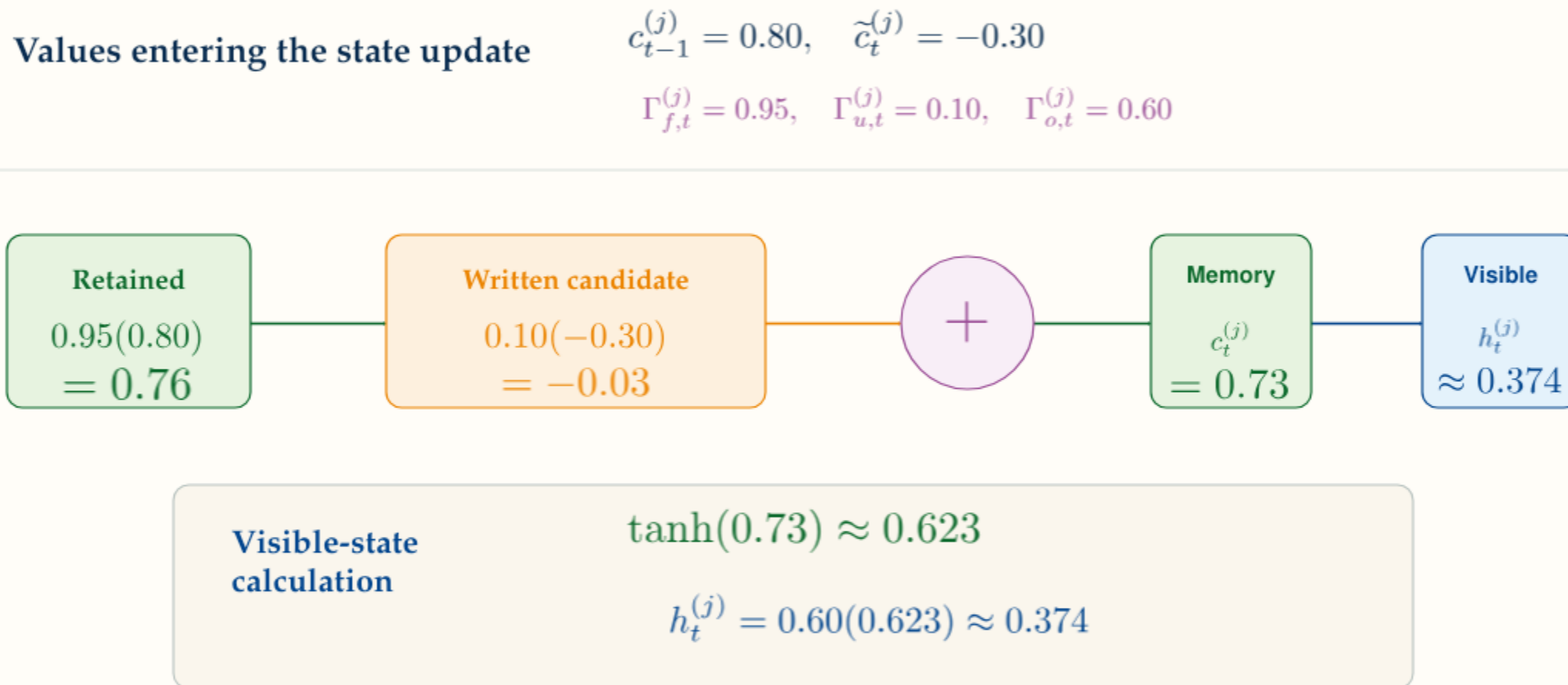


A scalar trace shows how the vector equations act on one memory coordinate.

A Numerical Trace Through One LSTM Coordinate

Every coordinate performs the same sequence of scalar calculations in parallel.

ONE COORDINATE OF THE RECURRENT STATE



Interpretation

The old memory contributes 0.76.

The candidate changes the coordinate by -0.03 .

The output gate exposes 60% of the transformed memory.

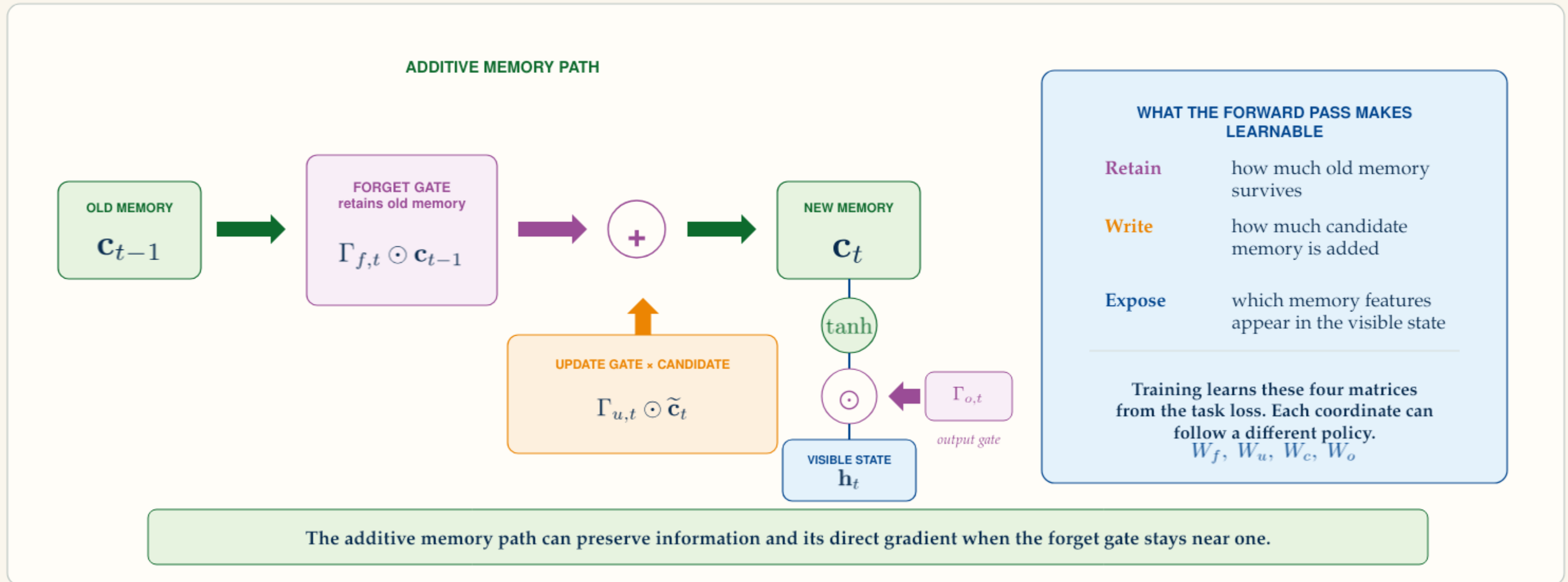
All state coordinates perform these calculations in parallel.

The additive memory update also creates a direct route for gradients through time.

LSTM: Trainable Memory Management

The forward pass provides a persistent memory path and learns how to retain, write, and expose information.

THE ARCHITECTURE SEPARATES MEMORY FROM THE VISIBLE STATE



RNN, GRU, and LSTM differ in the mechanism we propose. Training learns the numerical behaviour of each mechanism.